

Day2 实训手册：热点笔记查询 — Redis 缓存

一、今日任务目标

目标	说明
理解缓存的价值	热点数据读请求不再打到 MySQL
认识 Redis	掌握数据结构、常用命令与过期机制
掌握 Cache Aside 模式	读：先缓存后库并回填；写：先库后删缓存
完成改造	<code>GET /api/notes/{id}</code> 接入 Redis
产出数据	优化前后 QPS / RT / MySQL 压力对比
了解缓存三大问题	穿透 / 击穿 / 雪崩（今天认识，Day5 解决）
课堂实战	独立完成评论列表的缓存改造（第八节，课内启动、课下完成）

二、问题场景（先复现，再优化）

假设笔记 1 成为爆款，10000 个用户同时打开它：

1 | 优化前：10000 请求 → 10000 次 MySQL 查询（联表）

复现步骤：

1. 用 JMeter 对 `GET /api/notes/1` 压测：并发 300，持续 60 秒；
2. 观察：后端控制台 SQL 刷屏、`docker stats xhs-mysql` CPU 上升；
3. 记录优化前数据（填入第七节的表）。

三、Redis 基础知识（动手前先认识它）

在开始写代码之前，先花一点时间真正认识 Redis：它是什么、为什么快、数据怎么组织、生产环境里又是怎么部署的。这些知识不只是“背景介绍”，后面 7 天的每一个方案（计数、幂等、排行榜、限流）都直接建立在今天的内容之上。

3.1 Redis 是什么

Redis (Remote Dictionary Server) 是一个开源的**基于内存的键值数据库**。注意两个关键词：“内存”决定了它的速度，“键值”决定了它的使用方式——你不能像 MySQL 那样随意 JOIN、随意条件查询，而是用 Key 精确地存取 Value。这一特点正是它快的原因，也是后面“缓存设计”要围绕 Key 做文章的原因。

特点	说明
内存存储	数据放在内存中，读写延迟微秒级（MySQL 是毫秒级，相差约 1000 倍）
单线程模型	命令执行串行， 天然无并发竞态 ，INCR/SADD 等操作原子安全
丰富数据结构	不只是 String，还有 List/Set/ZSet/Hash 等（后面几天都会用到）
持久化	支持 RDB 快照 + AOF 日志，重启后数据可恢复
过期机制	每个 Key 可设置 TTL，到期自动删除（缓存场景的刚需）
单实例约 10 万 QPS	单机即可承接今天压测的全部读流量

一句话：Redis 是一个“放在内存里、数据结构丰富、快到离谱”的 Key-Value 存储，是高并发系统里 MySQL 前面的“挡箭牌”。

为什么不能直接给 MySQL 加内存来解决？ 经常有同学这么问。区别在于：MySQL 的慢不只是因为磁盘，还因为它要维护事务日志、复杂索引结构、锁机制等一整套关系型能力；而 Redis 从设计上就为“简单读写”而生，没有这些开销。打个比方：MySQL 是全能选手，什么都能干但背的装备多；Redis 是短跑运动员，只干一件事但干得极快。**两者不是替代关系，而是分工关系**：MySQL 存“真相”，Redis 存“热点”——这个分工思想会贯穿整个实训。

3.2 五大基础数据结构（本课程全部会用到）

Redis 不是只能存字符串的“大号 Map”，它内置了五种数据结构，这是它区别于 Memcached 等早期缓存的最大优势。**选型的核心原则：让数据的“形状”匹配结构的“能力”**——要存关系就用 Set，要排序就用 ZSet，不要把所有东西都塞进 String。

类型	特点	典型命令	在本项目中的用途
String	最基础，可存字符串/数字	SET / GET / INCR / EXPIRE	笔记详情缓存、点赞计数（今天）
Set	无序集合，元素唯一	SADD / SREM / SISMEMBER	点赞/收藏关系（Day3/4）
ZSet	有序集合，每个元素带 score	ZADD / ZINCRBY / ZREVRANGE	Feed 流、热榜（Day7）
Hash	字段-值映射，适合存对象	HSET / HGET / HGETALL	了解即可
List	双向队列	LPUSH / RPOP / LRange	了解即可

3.3 动手：redis-cli 实操

进入 Redis 容器命令行：

```
1 docker exec -it xhs-redis redis-cli
```

跟着敲一遍下面 5 组命令，理解 String 与过期机制：

```
1 # ① String: 设置与读取 (EX 120 表示 120 秒后过期)
2 SET demo:hello "world" EX 120
3 GET demo:hello
4
5 # ② 查看剩余存活时间 (秒)
```

```
6 TTL demo:hello
7
8 # ③ 计数器: INCR 原子自增 (高并发下不会丢数)
9 INCR demo:counter
10 INCR demo:counter
11 GET demo:counter
12
13 # ④ Set: 集合添加与判断成员 (点赞关系的核心命令)
14 SADD demo:likes "user:1" "user:2"
15 SISMEMBER demo:likes "user:1"
16 SISMEMBER demo:likes "user:999"
17
18 # ⑤ 清理演示数据 (注意: 不要在生产上执行 KEYS)
19 DEL demo:hello demo:counter demo:likes
```

课堂讨论: 为什么 INCR 在高并发下也不会数丢?
(单线程模型 → 每条命令原子执行, 不需要应用层加锁。这就是 Day3/4 用 Redis 承接高频写的底气。)

3.4 过期策略与内存淘汰 (理解 TTL 的底层逻辑)

给 Key 设置 TTL 之后, Redis 到底什么时候把它删掉? 内存不够用了又怎么办? 这两个问题决定了缓存的可靠性边界, 也是后面“缓存雪崩”话题的底层原理。

过期 Key 的删除时机 (两种配合使用) :

- **惰性删除**: 访问 Key 时才检查是否过期;
- **定期删除**: Redis 周期性抽查一批设置了 TTL 的 Key, 删除已过期的。

内存写满时的淘汰策略 (`maxmemory-policy`, 常见 4 种) :

策略	行为
<code>noeviction</code> (默认)	不淘汰, 写入直接报错
<code>allkeys-lru</code>	从所有 Key 中淘汰最近最少使用的 (缓存场景最常用)
<code>volatile-lru</code>	只在设置了 TTL 的 Key 中淘汰
<code>allkeys-random</code>	随机淘汰

结论: 缓存必须设置 TTL。

一方面兜底保证数据最终一致, 另一方面内存不够时, 带 TTL 的缓存会被优先淘汰, 不会挤占关键数据。

查看当前策略: `CONFIG GET maxmemory-policy`

3.5 生产环境的 Redis 部署方式 (认识篇)

今天课堂用的是 `docker compose` 起的**单实例** Redis——最简单, 但也最脆弱: 进程一挂, 缓存全部丢失, 请求瞬间全打到数据库。

真实的互联网公司不会这么用。下面介绍三种生产环境常见的部署形态, 它们是一步步演进出来的, 每一步都在解决上一步的问题。

(1) 主从复制 (Master-Slave Replication) : 解决“单点故障”与“读能力不足”

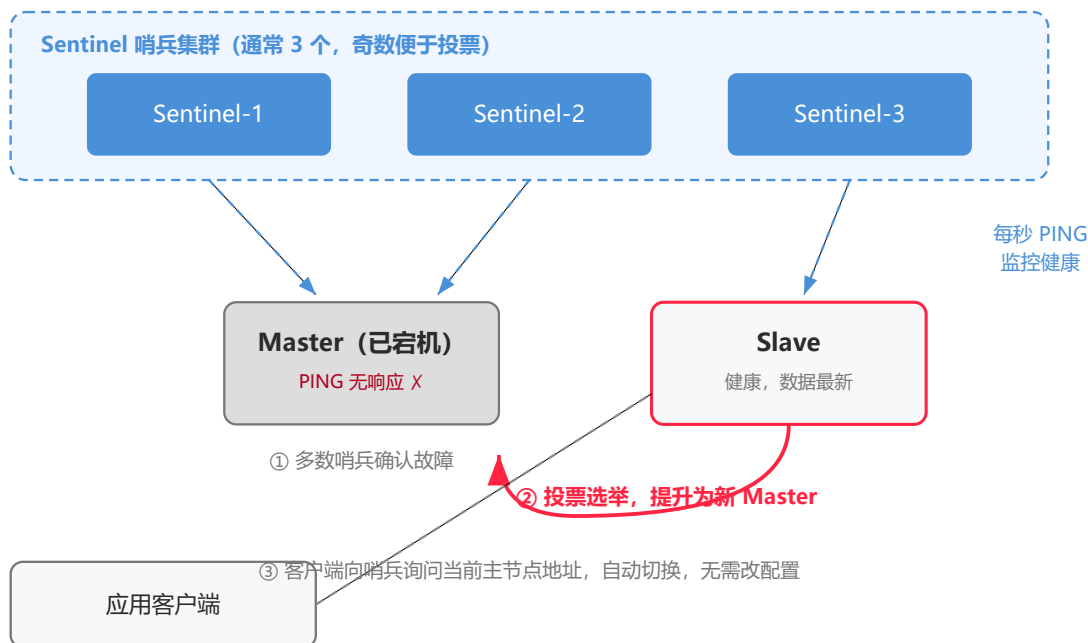
Redis 主从复制架构 (Master-Slave Replication)



核心思想: 写只走主节点, 主节点把数据异步复制给从节点, 读请求分摊到从节点。好处有两个: 一是读能力可以随从节点数量水平扩展 (这正是“读多写少”的缓存场景最需要的); 二是从节点保有全量数据, 主节点磁盘损坏时可从从节点恢复。但它的局限也很明显: **主节点宕机后需要人工介入切换**, 凌晨三点告警时没人愿意手动改配置; 而且异步复制意味着主节点挂的瞬间可能丢少量未同步的数据。

(2) 哨兵模式 (Sentinel) : 解决“自动故障转移”

Redis 哨兵模式 (Sentinel) : 自动故障转移



在主从的基础上再部署一组 Sentinel 进程 (通常 3 个, 奇数便于投票), 它们每秒向所有数据节点发 PING:

1. 多数哨兵确认主节点无响应 (客观下线);

2. 哨兵之间投票，把数据最新的从节点提升为新主节点；
3. 客户端向哨兵询问“现在的主节点是谁”，自动切换，**应用代码不用改、配置不用改**。

这套方案解决了“可用性”问题，也是 Spring Boot 的 Lettuce 客户端原生支持的模式（配置里填哨兵地址即可）。但它的天花板是：**单主节点的内存和写入能力无法突破**—数据量超过单机内存时，就需要第三种形态。

(3) 集群模式 (Redis Cluster)：解决“容量与吞吐的天花板”

Redis Cluster 集群：数据分片 + 水平扩容



Cluster 把全部数据空间划分为 **16384 个槽位 (slot)**，每个主节点负责其中一段：写入时客户端对 Key 计算 $\text{CRC16}(\text{key}) \% 16384$ ，就能算出数据该去哪个节点。没有中心代理，节点之间通过 gossip 协议互通状态；每个主节点配一个从节点做备份，某个主节点挂了，它的从节点自动顶替（故障转移内置在集群里）。扩容也很直接：新加节点、迁移一部分槽位过去，容量和吞吐随节点数线性增长。代价是使用复杂度上升：多键操作（如 MGET 跨槽位）受限、运维成本更高，所以**数据量没到单机极限前一般不上 Cluster**。

四种形态怎么选？

部署方式	解决什么问题	适用规模	代价
单实例	—	开发/课堂/小规模	无高可用，宕机即丢缓存
主从复制	读扩展、数据备份	读多写少的中小系统	故障切换靠人工
哨兵模式	自动故障转移	要求高可用的主流选择	单主节点容量有上限
Cluster 集群	容量与吞吐水平扩展	海量数据/超高并发	使用与运维复杂

本课的对应关系：课堂环境用单实例（够用即可，避免复杂度干扰学习重点）；如果你的实训项目要参加答辩/演示“生产化”，把 docker compose 里的 Redis 换成“1 主 1 从 + 3 哨兵”就是一个很好的加分项；Day8 的“降级”方案（Redis 挂了回退 MySQL）则是应用层对所有部署形态都适用的保险丝。

面试常问：主从复制是同步还是异步？哨兵怎么选新主？Cluster 为什么是 16384 个槽？（课后自查，答案都能在本节找到线索）

四、方案设计

4.1 Cache Aside 读流程

```
1  请求 GET /notes/{id}
2      ↓
3  查 Redis (key = note:{id})
4      └─ 命中 → 直接返回
5      └─ 未命中
6          ↓
7          查 MySQL
8          ↓
9          写回 Redis (设置 TTL)
10         ↓
11        返回
```

4.2 关键设计

设计点	决策	理由
Key 设计	<code>note:{noteId}</code>	业务前缀 + 主键，避免冲突
Value	NoteVO 的 JSON (含类型信息)	一次反序列化即可返回
TTL	30 分钟	兜底保证最终一致；Day5 会加随机抖动
序列化	Jackson	支持 <code>LocalDateTime</code> 等复杂对象

五、编码实现

按 `code/day2/` 目录完成以下操作：

序号	文件	操作
1	<code>pom.xml</code>	【替换】新增 <code>spring-boot-starter-data-redis</code> 依赖
2	<code>application.yml</code>	【替换】新增 <code>redis</code> 连接配置
3	<code>config/RedisConfig.java</code>	【新增】 <code>RedisTemplate</code> 序列化配置
4	<code>common/RedisKeys.java</code>	【新增】统一管理所有 <code>Redis Key</code>
5	<code>service/NoteService.java</code>	【替换】 <code>detail</code> 方法接入缓存

核心代码 (`NoteService.detail`)：

```
1  public NoteVO detail(Long id, Long viewerId) {
2      String key = RedisKeys.note(id);
3      // 1. 先查缓存
```

```
4     NoteVO vo = (NoteVO) redisTemplate.opsForValue().get(key);
5     // 2. 未命中 → 查库 → 回填
6     if (vo == null) {
7         vo = noteMapper.selectDetail(id);
8         if (vo != null) {
9             redisTemplate.opsForValue().set(key, vo, 30, TimeUnit.MINUTES);
10        }
11    }
12    if (vo == null) {
13        return null;
14    }
15    fillStatus(Collections.singletonList(vo), viewerId);
16    return vo;
17 }
```

六、验证与压测

6.1 功能验证

- 1. 重启后端，访问笔记详情，确认正常显示；
- 2. 用 Redis 客户端执行 `GET note:1`，确认有 JSON 数据、`TTL note:1` 有剩余时间；
- 3. 观察后端控制台：第二次访问**不再打印 SQL**。

6.2 压测对比（与第二节相同条件）

观察项	优化前（基线）	优化后
压测接口	GET /api/notes/1	GET /api/notes/1
QPS		
平均 RT		
P95		
MySQL CPU (docker stats)		
后端 SQL 日志	刷屏	仅首次一条

七、结果记录表

指标	优化前	优化后	变化
QPS			
平均 RT(ms)			
P95(ms)			
错误率			
MySQL CPU 峰值			

八、课堂实战：给评论列表加缓存（自己动手）

照着今天学到的 Cache Aside 套路，独立完成另一个接口的缓存改造。
这条任务链会贯穿后面几天：今天做缓存，Day4 给它加幂等，Day6 给它做异步落库，请尽量当天完成。

需求：评论列表接口 `GET /api/notes/{noteId}/comments` 目前每次都查 MySQL，请把它接入 Redis：

- Key 设计：`comment:list:{noteId}`，Value 存评论列表 JSON，TTL 设为 5 分钟；
- 缓存失效：发新评论成功后，删除对应缓存（先写库、后删缓存）；
- Key 统一登记在 `RedisKeys.java`。

实现步骤：

1. `common/RedisKeys.java` 新增方法：`public static String commentList(Long noteId) { return "comment:list:" + noteId; }`；
2. 参照 `NoteService.detail` 的三段式结构改造 `CommentService.list()`：查缓存 → 未命中查库并回填 → 返回；
3. 在 `CommentService.add()` 中，评论入库成功后执行 `redisTemplate.delete(RedisKeys.commentList(noteId))`；
4. 重启验证。

关键代码提示（list 方法，仿照 detail）：

```
1 public List<CommentVO> list(Long noteId) {
2     String key = RedisKeys.commentList(noteId);
3     List<CommentVO> list = (List<CommentVO>) redisTemplate.opsForValue().get(key);
4     if (list == null) {
5         list = commentMapper.selectByNoteId(noteId); // 方法名以实际 Mapper 为准
6         redisTemplate.opsForValue().set(key, list, 5, TimeUnit.MINUTES);
7     }
8     return list;
9 }
```

验收标准（全部打勾才算完成）：

连续两次访问同一笔记的评论列表，控制台 SQL 只打印一次；

`redis-cli` 执行 `GET comment:list:1` 有 JSON，TTL `comment:list:1` 有剩余秒数；

发表一条新评论后，再访问列表能立刻看到新评论（证明删缓存生效）。

选做挑战：如果某篇笔记评论量极大（上万条），整列表缓存会有什么问题？你会怎么改？（提示：只缓存第一页 / 分页缓存）

九、缓存三大问题：穿透 / 击穿 / 雪崩（认识篇）

今天先把概念建立起来，**解决方案在 Day5 完整落地：**

问题	场景	后果	解决思路 (Day5)
缓存穿透	查询 根本不存在 的数据 (如 id=-1) , 缓存永远查不到	请求全部打到数据库	空对象缓存 / 布隆过滤器
缓存击穿	热点 Key 过期瞬间, 大量并发同时回源	数据库瞬间被压垮	互斥锁 (分布式锁) 重建
缓存雪崩	大量 Key 同时过期, 或 Redis 宕机	请求集中打到数据库	TTL 随机抖动 / 多级缓存

记忆口诀：穿透查的是“不存在的”，击穿挂的是“最热的”，雪崩倒的是“一大片”。

十、思考题

- 1. 如果笔记被修改了，缓存里的旧数据怎么办？（提示：写操作时删除缓存）
- 2. 为什么 TTL 不能设为“永不过期”？（结合 3.4 的淘汰策略回答）
- 3. 用 redis-cli 执行 `TTL note:1`，观察过期时间的变化；再用 `TYPE note:1` 查看它的数据类型。
- 4. 如果 1000 个请求同时发现缓存未命中，会发生什么？（对照第九节，说出这是哪类问题）